

Como adicionar um módulo ao Morphe

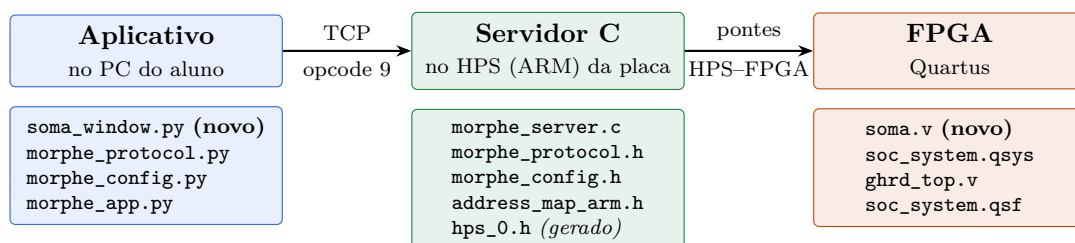
Roteiro prático — exemplo: a soma de dois sinais

Antonio Nicassio Santos Lima Estágio LabPS/UEFS — supervisão: Prof. Armando S. Sanca

outubro de 2026

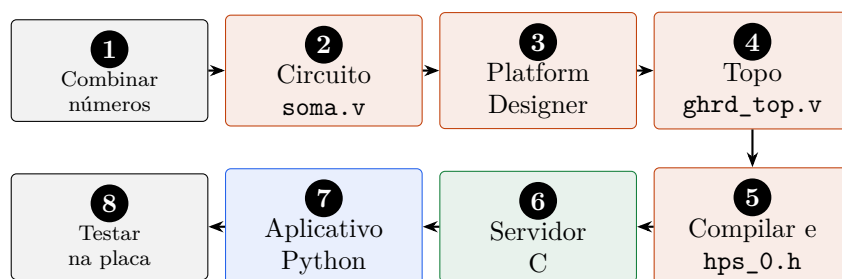
O exemplo é $y[n] = a[n] + b[n]$, com até 1024 amostras em Q15.16 (o formato de todo o Morphe). A conta é trivial de propósito: o que importa é o caminho, que é o mesmo para qualquer acelerador. Todo o código está pronto na branch `estagio/modulo-soma` — use-o como modelo.

O mapa: o que muda em cada camada



A ordem de construção

De baixo para cima: cada camada só é escrita quando a de baixo já existe, e assim um erro fica na camada que acabou de entrar.

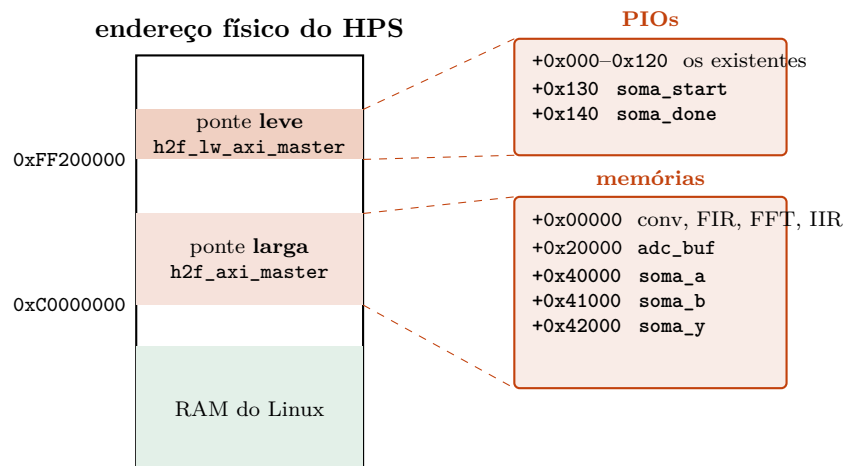


1 Combinar os números

Os mesmos números aparecem em vários arquivos. Anote antes de começar:

O quê	Valor	Onde aparece
opcode	9	<code>morphe_protocol.h</code> e <code>morphe_protocol.py</code>
amostras	1024	<code>morphe_config.h</code> , <code>morphe_config.py</code> , <code>N_SAMPLES</code> do Verilog
formato do pedido	o da convolução	cabeçalho + $a + b$ (seção 7)
memórias a , b , y	4 KiB cada	0x40000, 0x41000, 0x42000
PIOs <code>start</code> , <code>done</code>	1 bit cada	0x130, 0x140

Como o HPS enxerga a FPGA



- O endereço do Platform Designer é um **deslocamento** dentro da ponte: `soma_a` está em $0xC0000000 + 0x40000$.
- Memórias vão na ponte larga; PIOs (registradores de controle), na leve.
- **Onde ver o que está ocupado:** no `C/hps_0.h`, cada componente tem um `_BASE` (deslocamento) e um `_SPAN` (tamanho). Antes da soma, a ponte larga ia até `0x3FFFF` e o último PIO era `0x120`.

```
grep -E "_ (BASE|SPAN)" C/hps_0.h | sort -k3
```

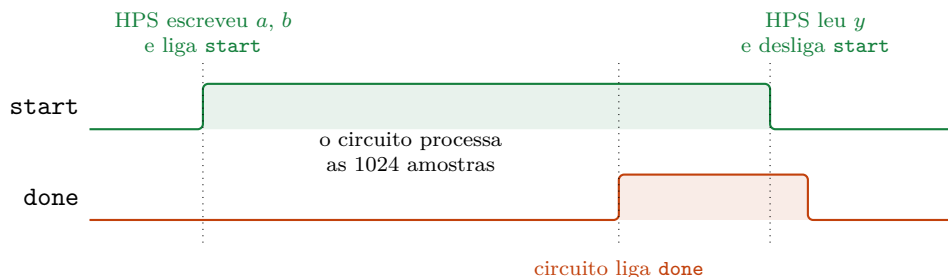
2 O circuito: soma.v

Faça

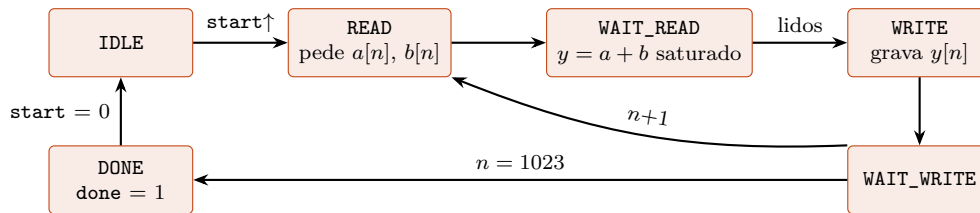
- Crie `Quartus/soma.v` — na **raiz** de `Quartus/`, ao lado do `conv1d.v`.
- Ponha no projeto, no `Quartus/soc_system.qsf` (ou em *Project* → *Add/Remove Files*):

```
set_global_assignment -name VERILOG_FILE soma.v
```

Todo acelerador do Morphe conversa com o HPS do mesmo jeito:



Por dentro, uma máquina de seis estados:



Já existe: reaproveite

Ler e escrever as memórias já está feito: `memory_read_controller.v` e `memory_write_controller.v` (os mesmos do `conv1d.v` e do `iir_cascade.v`). Dê o endereço e um pulso em `start_read`; o dado volta em `data_out` com um pulso em `read_done`. A soma usa dois de leitura (a e b em paralelo) e um de escrita (y).

```

memory_read_controller #(.DATA_WIDTH(32), .MEM_ADDRESS_N_BITS(10)) u_rd_a (
    .clk(clk), .reset_n(reset_n),
    .start_read(rd_start),          .data_addr(n),
    .sram_readdata(a_sram_readdata), .data_out(a_data), .read_done(a_done),
    .mem_address(a_sram_address),    .mem_write(a_sram_write),
    .clken(a_sram_clken),            .chipselect(a_sram_chipselect));
  
```

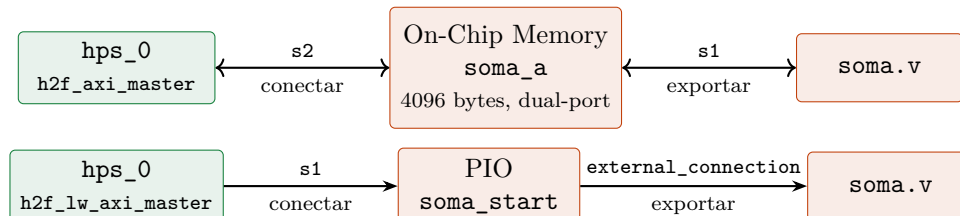
A conta é uma linha: soma em 33 bits e satura em 32, para um valor grande não “dar a volta” e virar negativo.

```

wire signed [32:0] soma_larga = $signed({a_data[31], a_data}) + $signed({b_data[31], b_data});
wire [31:0] soma_sat = (soma_larga > MAXV) ? MAXV : (soma_larga < MINV) ? MINV : soma_larga[31:0];
  
```

3 Platform Designer: memórias e PIOs

Cada memória tem **duas portas**: uma para o HPS, outra para o circuito.



Faça

Quartus 20.1 → *Tools* → *Platform Designer* → abrir `Quartus/soc_system.qsys`.

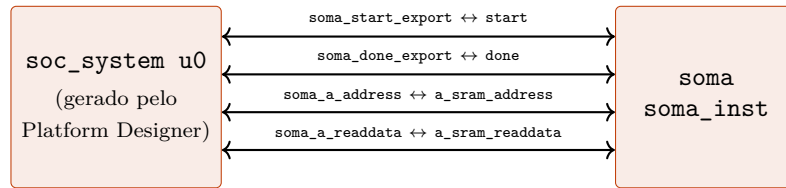
1. **3 memórias** (`soma_a`, `soma_b`, `soma_y`): *On-Chip Memory (RAM or ROM) Intel FPGA IP*; RAM, *Dual-port access* marcado, 32 bits, 4096 bytes, M10K.
2. **2 PIOs**: *PIO (Parallel I/O) Intel FPGA IP*, 1 bit; `soma_start` *Output*, `soma_done` *Input*.
3. Ligar como no diagrama; `clk` em `clk_0.clk` e `reset` em `clk_0.clk_reset`.
4. Na coluna *Base*, digitar os endereços da seção 1 e travar.
5. Salvar e **gerar** (seção 5).

Cuidado

Não use *System* → *Assign Base Addresses*: ele muda os endereços de *todos* os componentes, e o servidor e as placas deixam de concordar. Endereços novos à mão, nos buracos livres.

4 O topo: ghrd_top.v

O Quartus/ghrd_top.v liga o sistema do Platform Designer (u0) ao circuito (soma_inst) por fios:



... um fio por sinal: 2 dos PIOs + 7 por memória

Faça

Quatro inserções, cada uma ao lado do bloco equivalente do IIR:

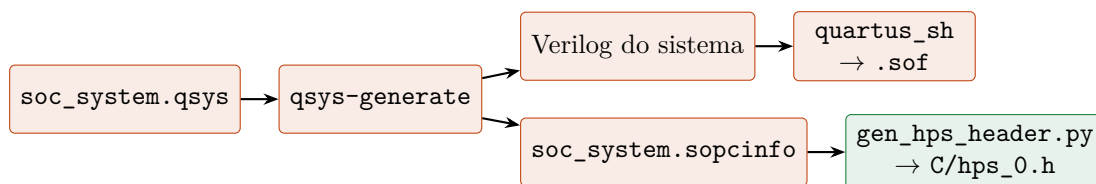
1. localparam SOMA_DATA_WIDTH, SOMA_ADDR_WIDTH, SOMA_N_SAMPLES;
2. os wire: soma_start, soma_done e 7 por memória;
3. a instância soma #(...) soma_inst (...);
4. as portas novas na instância u0.

```
// em u0 (soc_system):
.soma_start_export (soma_start),
.soma_done_export (soma_done),
.soma_a_address (soma_a_address),
.soma_a_readdata (soma_a_readdata),
// ... clken, chipselect, write, writedata, byteenable, e o mesmo para b e y
```

Confira

Os nomes das portas estão no começo de Quartus/soc_system/synthesis/soc_system.v, depois de gerar: `grep -n soma Quartus/soc_system/synthesis/soc_system.v`. Não use o soc_system_inst.v: no repositório ele está desatualizado.

5 Compilar e gerar o hps_0.h



Faça

Na estação, com o Quartus 20.1, de dentro de Quartus/:

```
qsys-generate soc_system.qsys --synthesis=VERILOG --family="Cyclone V" --part=5CSEMA5F31C6
python3 gen_hps_header.py
quartus_sh --flow compile soc_system
```

Confira

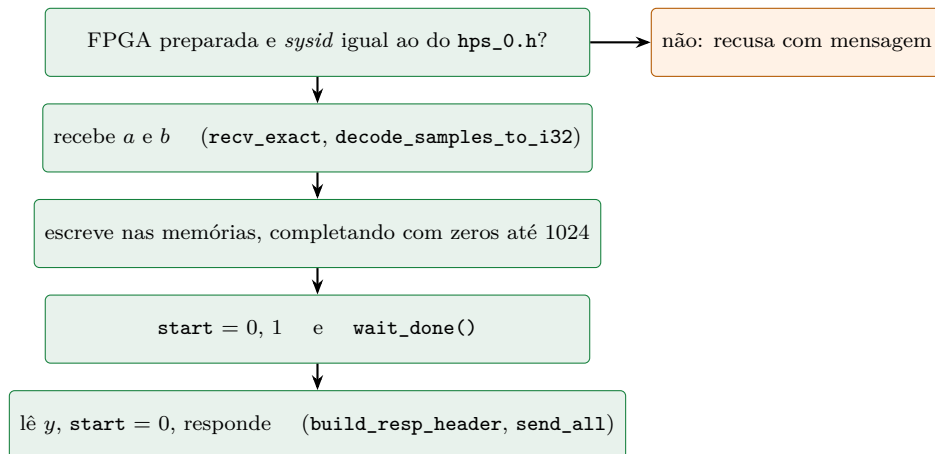
- `grep SOMA_ ../C/hps_0.h` mostra os 5 endereços da seção 1;
- *Full Compilation was successful*;
- em `output_files/soc_system.sta.summary`, nenhum *Slack* negativo no `clock_50_1`.

Cuidado

Nunca edite o `hps_0.h` à mão, e gere sempre com a 20.1. Se `qsys-generate` não for encontrado, o terminal está em `ksh`: digite `bash`.

6 O servidor C

O que a função nova, `handle_soma()`, faz — o molde é a `handle_fir()`, que tem o mesmo formato:



Acessar a FPGA é usar um ponteiro: o início da janela da ponte (já mapeada pelo servidor) mais o deslocamento do `hps_0.h`.

```
g_soma_a      = (int32_t *)((char *)g_fpga_virt + SOMA_A_BASE);      /* ponte larga */
g_pio_soma_start = (volatile uint32_t *)((char *)g_lw_virt + SOMA_START_BASE); /* ponte leve */

g_soma_a[i] = valor;          /* escreve a palavra i da memoria */
*g_pio_soma_start = 1;        /* liga o start */
```

Faça

```
morphe_config.h    #define MORPHE_SOMA_N_MAX 1024
morphe_protocol.h  #define MORPHE_OP_SOMA 9U
address_map_arm.h  FPGA_ONCHIP_SPAN de 0x3FFFF para 0x4FFFF (a janela tem de alcançar 0x42FFF)
morphe_server.c    MORPHE_TEM_SOMA: só compila a soma se o hps_0.h a tem
                   os 5 ponteiros, na fpga_init()
                   a handle_soma()
                   case MORPHE_OP_SOMA: no despacho e "soma" em nome_op()
```

Já existe: reaproveite

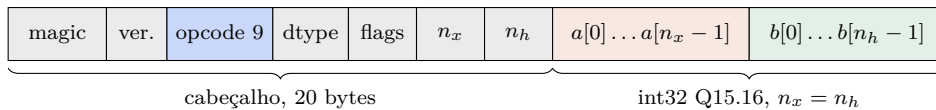
`recv_exact`, `send_all`, `send_error`, `decode_samples_to_i32`, `wait_done`, `build_resp_header` e `save_debug_bundle_conv` já estão no servidor. E a proteção do `sysid`, feita para o ADC: o servidor só toca na soma se o bitstream gravado na placa é o mesmo do seu `hps_0.h`.

Cuidado

Acessar um endereço da ponte **onde não há componente trava a placa** (ela some da rede). Por isso: nada de acesso à FPGA na `fpga_init()`, e a soma só com o `sysid` conferido. O servidor não precisa ser compilado à mão: o `morphe-up.sh` envia e recompila na placa.

7 O aplicativo

O pedido tem o mesmo formato da convolução:



Faça

```
morphe_config.py    SOMA_N_MAX = 1024
morphe_protocol.py  OP_SOMA = 9, build_soma_request(), decode_soma_response()
soma_window.py      a janela (novo arquivo)
morphe_app.py        o botão: um import, uma linha na lista e _open_soma()
```

```
def build_soma_request(a_q, b_q, dtype_code=DTYPE_INT32) -> bytes:
    hdr = struct.pack(">IH HHII", MAGIC_REQ, VERSION, OP_SOMA, dtype_code, 0,
                      len(a_q), len(b_q))
    return hdr + pack_samples(a_q, dtype_code) + pack_samples(b_q, dtype_code)
```

Já existe: reaproveite

A janela é curta porque quase tudo existe: `SignalPanel` (o painel de sinal), `_stem_signal` (o gráfico da janela de convolução), `float_to_q1516` e `q1516_to_float` (`dsp_core.py`), `self.master.tcp_panel.make_client()` (o cliente já apontado para a placa escolhida) e o tema.

Cuidado

O pedido vai numa *thread* separada, senão a janela congela esperando a rede; o resultado volta à interface com `self.after(0, ...)`.

8 Testar na placa

Faça

Do clone, na estação, fora do horário de aula:

```
./morphe-up.sh --cable 'DE-SoC [1-2]' --deploy
python3 Python/morphe_app.py
```

Depois, para devolver a placa à turma, `./morphe-up.sh` de `/opt/morphe` com o mesmo cabo.

Confira

- os 4 testes do `morphe_ping` passam (o resto continua funcionando);
- na janela Soma, a barra de status diz que as amostras são **iguais, bit a bit**, à conta feita no NumPy;
- $a = b = 30\,000$ dá 32 767,99998 (saturou), não um negativo;
- numa placa com o bitstream antigo, a soma é **recusada** com mensagem — e a placa não trava.

Por fim: proveniência (`./gera_proveniencia.sh`), `commit` do `.sof`, `.sopcinfo`, `hps_0.h` e `soc_system/`, mesclar na principal, `/opt/morphe/atualizar.sh` nos computadores e `./morphe-up.sh --todas` na estação.

Erros comuns

Sintoma	Causa e solução
o Quartus não acha o módulo	<code>.v</code> fora da raiz de <code>Quartus/</code> ou fora do <code>.qsf</code>
operações antigas param de funcionar	usou <i>Assign Base Addresses</i> : os endereços mudaram
arquivos mudados sem relação com o módulo	gerou com outra versão do Quartus; use a 20.1
<code>_Static_assert</code> falha ao compilar o servidor	esqueceu o <code>FPGA_ONCHIP_SPAN</code>
a placa some da rede	acesso a endereço sem componente: <code>hps_0.h</code> diferente do bitstream
a placa para depois de cerca de uma hora	sem o <i>tether</i> da licença; use o <code>morphe-up.sh</code> , sem <code>sudo</code>
<code>qsys-generate: not found</code>	terminal em <code>ksh</code> ; digite <code>bash</code>